

Algoritmi e Strutture Dati – Parte A – 12/01/26

Esercizio -1 Iscriverti allo scritto entro la scadenza.
In caso di inadempienza, -1 al voto finale.

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su tutti i fogli consegnati. Consegnare foglio A4 e foglio protocollo di bella. In caso di inadempienza, -1 al voto finale.

A1 – Complessità – Punti ≥ 8

Scrivere l'equazione di ricorrenza associata a questo algoritmo e calcolare la complessità computazionale che ne deriva.

```

int mystery(int[] A, int start, int end)
{
    if end == start then
        return A[start]
    else
        int i = end - start
        int sumSoFar = 0
        while i > 0 do
            sumSoFar += A[start + i]
            i = i/2
        int mid = ⌊(start + end)/2⌋
        sumSoFar += mystery(A, start, mid)
        sumSoFar += mystery(A, mid + 1, end)
        return sumSoFar
}

```

A2 – Alberobello – Punti ≥ 10

Un *alberobello* è un albero binario in cui ogni nodo ha esattamente due figli oppure nessun figlio.

Dato un albero binario T , diciamo che un nodo t di T è una *radice alberobella* se il sottoalbero di T radicato in t è un alberobello, cioè se *tutti* i nodi di tale sottoalbero hanno zero o due figli.

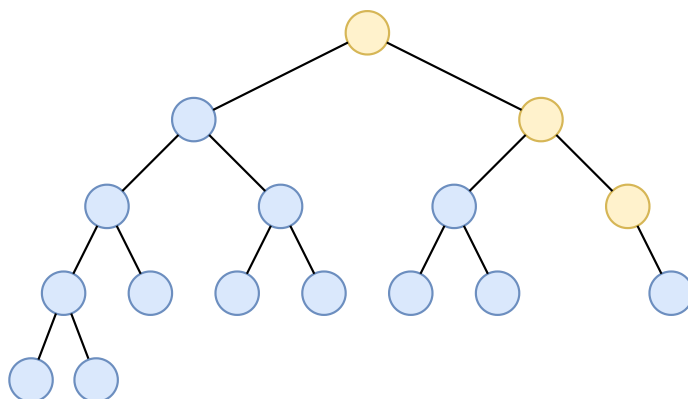
Scrivere un algoritmo

```
int countAB(TREE T)
```

che restituisca il numero di nodi di T che sono radici alberobelle.

Discutere informalmente la correttezza dell'algoritmo proposto e la sua complessità computazionale.

Per esempio, nell'albero seguente, tutti i nodi evidenziati in azzurro sono radici alberobelle; pertanto, l'algoritmo deve restituire il valore 13.



A3 – Massimo raggiungibile – Punti ≥ 12

Scrivere un algoritmo

```
int[] maxReachable(GRAPH G)
```

che prende in input un grafo non orientato G , dove ogni nodo è dotato di un campo *value* che contiene un valore intero. L'algoritmo deve restituire un vettore M tale che, per ogni nodo u , $M[u]$ sia il massimo valore associato a un nodo v raggiungibile da u in G .

Si ricorda che un nodo v è *raggiungibile* da un nodo u se esiste un cammino in G che parte da u e termina in v .

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale.